Classes & Objects

A **class** is a blueprint; an **object** is one thing built from it.

1) Create a class and objects

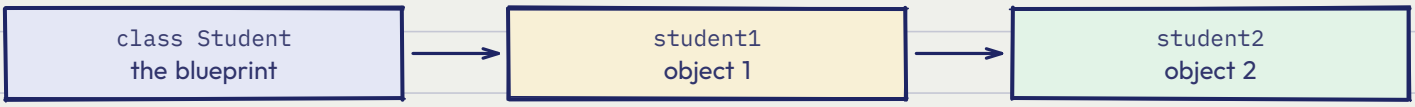
Example :

```
class Student:
    pass

student1 = Student()
student2 = Student()
student1.name = "Amit"
student2.name = "Priya"
print(student1.name)
print(student2.name)
```

Output :

```
Amit
Priya
```



this works, but adding attributes by hand gets repetitive. __init__ fixes that later

2) Class attributes vs instance attributes

Example :

```
class Employee:
    company = "ABC Ltd"
    branch_name = "Biomedical"

employee1 = Employee()
employee2 = Employee()
employee1.company = "Local Ltd"
employee2.branch_name = "Electronics"
print("Employee 1 company:", employee1.company)
print("Employee 2 company:", employee2.company)
print("Employee 2 branch:", employee2.branch_name)
print("Class value:", Employee.company)
```

Output :

```
Employee 1 company: Local Ltd
Employee 2 company: ABC Ltd
Employee 2 branch: Electronics
Class value: ABC Ltd
```

☆ REMEMBER

- A **class attribute** is a shared default for every object.
- An **instance attribute** belongs to one object; changing it never changes the class value.

METHODS, SELF & __INIT__

A **method** is a function inside a class. `self` is the object that is calling it.

1) Methods and self

Example :

```
class Department:
    def introduce(self):
        print("This is the Data Science Department.")
        print("We work with data and machine learning.")

professor1 = Department()
professor1.introduce()
```

Output :

```
This is the Data Science Department.
We work with data and machine learning.
```

professor1.introduce() passes professor1 to self automatically

2) __init__() sets up each object

Example :

```
class Employee:
    def __init__(self, name, salary, department):
        self.name = name
        self.salary = salary
        self.department = department

    def show_info(self):
        print(f"Employee name: {self.name}")
        print(f"Salary: Rs.{self.salary}")
        print(f"Department: {self.department}")

employee1 = Employee("Sanatan", 40000, "Electronics")
employee2 = Employee("Rahul", 45000, "Biomedical")
employee1.show_info()
employee2.show_info()
```

Output :

```
Employee name: Sanatan
Salary: Rs.40000
Department: Electronics
Employee name: Rahul
Salary: Rs.45000
Department: Biomedical
```

3) What each line means

Line	Meaning
`def __init__(self, name, salary)`	runs automatically when an object is created
`self.name = name`	stores the value on that specific object
`Employee("Sanatan", 40000, ...)`	creates an object and initialises its attributes

every object gets its own copy of name, salary and department

Encapsulation: Safe Updates

Encapsulation keeps data with the methods that change it, and checks values first.

1) Bank account with validation

Example :

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be greater than zero.")
        else:
            self.balance += amount
            print("Deposit successful.")

    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be greater than zero.")
        elif amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print("Withdrawal successful.")

    def show_balance(self):
        print(f"{self.account_holder}: Rs.{self.balance}")

account = BankAccount("Aarav", 20000)
account.deposit(3000)
account.withdraw(5000)
account.show_balance()
account.deposit(-500)
account.withdraw(50000)
account.show_balance()
```

Output :

```
Deposit successful.
Withdrawal successful.
Aarav: Rs.18000
Deposit amount must be greater than zero.
Insufficient funds.
Aarav: Rs.18000
```

☆ REMEMBER

- Methods such as `deposit()` and `withdraw()` check the amount **before** changing the balance.
- This is beginner encapsulation: outside code can still change `balance`, so always go through the methods.

Practice & Revision

Write each class yourself first, then compare. Each solution is run for real.

Task 1 Student with name and course.

Solution :

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course
    def show_info(self):
        print(f"Name: {self.name}")
        print(f"Course: {self.course}")
Student("Aarav", "Python").show_info()
```

```
Name: Aarav
Course: Python
```

Task 2 Rectangle area and perimeter.

Solution :

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
r = Rectangle(10, 5)
print("Area:", r.area())
print("Perimeter:", r.perimeter())
```

```
Area: 50
Perimeter: 30
```

Task 3 Marks only from 0 to 100.

Solution :

```
class Student:
    def __init__(self, name):
        self.name = name
        self.marks = 0
    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.marks = marks
        else:
            print("Marks must be between 0 and 100.")
s = Student("Aarav")
s.set_marks(105)
s.set_marks(92)
print(s.marks)
```

```
Marks must be between 0 and 100.
92
```

Task 4 Volume from 0 to 10.

Solution :

```
class VolumeControl:
    def __init__(self):
        self.volume = 5
    def set_volume(self, new_volume):
        if 0 <= new_volume <= 10:
            self.volume = new_volume
        else:
            print("Volume must be between 0 and 10.")
remote = VolumeControl()
remote.set_volume(8)
remote.set_volume(15)
print(remote.volume)
```

```
Volume must be between 0 and 10.
8
```

Quick revision

Term	Meaning
class / object	blueprint / one thing built from it
self	the object that is calling the method
__init__()	sets up each new object's attributes
encapsulation	data + methods together, values validated